Linearizing the GL-FLIF Reservoir: A Detailed Walkthrough

Supplement to "Neuron-Dropout Faults in Fractional-Order Reservoirs"

21 October 2026

Abstract

The main document analyses faults in a network of spiking neurons by first replacing the network with a linear model. This supplement explains that replacement step by step. It assumes calculus, Taylor expansion and finite-difference approximations of derivatives, and nothing about spiking neurons or fractional calculus. Section 1 builds the neuron model from finite differences. Section 2 explains why it is not linear. Sections 3–6 remove the obstacles one at a time, and Section 7 assembles the linear network. Section 8 says where the approximation can be trusted. The figures come from simulating a single neuron with the main document's update rule.

Contents

1 The neuron model **2**
- 1.1 A leaky integrate-and-fire neuron . . . 2
- 1.2 From finite differences to fractional differences . . . 2
- 1.3 The GL-FLIF update . . . 3
- 1.4 The network . . . 4

2 Why the model is not linear **4**

3 Step 1: the free potential **4**

4 Step 2: the operating point **5**

5 Step 3: a smooth spike response **6**
- 5.1 Averaging turns a step into a smooth curve . . . 6
- 5.2 The f–I curve . . . 6
- 5.3 The tangent line, in units of the free potential . . . 6

6 Measuring r_j, γ_j and $\bar{V}_j$ **7**

7 Step 4: assembling the linear network **9**
- 7.1 One neuron's equation . . . 9
- 7.2 A two-neuron example . . . 9
- 7.3 The general form . . . 10

8 When can the approximation be trusted? **10**

Symbols **11**

1 The neuron model

1.1 A leaky integrate-and-fire neuron

A model neuron has one state variable, its *membrane potential* $V(t)$, which plays the role of a voltage. Between events it obeys a linear ordinary differential equation:

$$\frac{dV}{dt} = -\frac{V - V_{\text{rest}}}{\tau_m} + I(t) + b. \tag{1}$$

The first term is a *leak*: on its own it pulls V back to the resting value V_{rest} with time constant τ_m. The second term is the *input current* $I(t)$, which pushes V up or down, and b is a constant bias. If the input is constant, V settles where the right-hand side vanishes, at $V = V_{\text{rest}} + \tau_m(I + b)$.

The nonlinearity comes from one extra rule. When V reaches a *threshold* V_{th}, the neuron emits a *spike* and V is immediately set back to a *reset* value V_{reset}. A spike is an all-or-nothing event, so the neuron's output at time step n is a binary number $s_n \in \{0, 1\}$. This is the *leaky integrate-and-fire* (LIF) neuron: it integrates its input, leaks, and fires.

Discretizing Eq. (1) with forward Euler and step Δt gives

$$V_n = V_{n-1} + \Delta t \left(-\frac{V_{n-1} - V_{\text{rest}}}{\tau_m} + I_n + b \right), \tag{2}$$

followed by the threshold check: if $V_n \geq V_{\text{th}}$, set $s_n = 1$ and $V_n = V_{\text{reset}}$; otherwise $s_n = 0$. The fractional neuron of the main document replaces the left-hand side derivative of Eq. (1) by a *fractional* derivative. The next subsection builds that derivative out of finite differences.

1.2 From finite differences to fractional differences

Write ∇ for the backward difference, $\nabla x_n = x_n - x_{n-1}$. The familiar approximations are

$$x'(t_n) \approx \frac{\nabla x_n}{\Delta t} = \frac{x_n - x_{n-1}}{\Delta t}, \qquad x''(t_n) \approx \frac{\nabla^2 x_n}{\Delta t^2} = \frac{x_n - 2x_{n-1} + x_{n-2}}{\Delta t^2}.$$

The weights $1, -1$ and $1, -2, 1$ are binomial coefficients with alternating signs. To see why, let B be the operator that shifts a sequence back one step, $Bx_n = x_{n-1}$. Then $\nabla = 1 - B$, and for any whole number m the binomial theorem gives

$$\nabla^m x_n = (1 - B)^m x_n = \sum_{k=0}^{m} (-1)^k \binom{m}{k} x_{n-k}.$$

Newton's generalized binomial theorem extends $(1 - B)^m$ to a non-integer power α. The binomial coefficient is defined for any real α by keeping only the falling product on top,

$$\binom{\alpha}{k} = \frac{\alpha(\alpha - 1) \cdots (\alpha - k + 1)}{k!}, \qquad k = 0, 1, 2, \ldots,$$

which is a polynomial in α and needs no factorial of a non-integer. The *fractional difference* of order α is then

$$\nabla^\alpha x_n = (1 - B)^\alpha x_n = \sum_{k=0}^{\infty} c_k \, x_{n-k}, \qquad c_k = (-1)^k \binom{\alpha}{k}, \tag{3}$$

and the *Grünwald–Letnikov (GL) derivative* is its limit, $D^\alpha x(t_n) = \lim_{\Delta t \to 0} \Delta t^{-\alpha} \, \nabla^\alpha x_n$. For $\alpha = 1$ and $\alpha = 2$ the sum stops after two or three terms and reproduces the ordinary differences. Powers of $(1 - B)$ multiply in the usual way, so applying the half difference $\nabla^{1/2}$ twice gives the ordinary first difference ∇.

The coefficients are easiest to compute from the ratio of consecutive terms, which follows directly from the product form:

$$c_0 = 1, \qquad c_k = c_{k-1}\left(1 - \frac{\alpha+1}{k}\right). \tag{4}$$

Table 1 lists the first few. The key difference from the integer case is visible in the last row. For non-integer α no coefficient is ever zero, so a fractional difference involves the *entire* past of the sequence. That is the long memory of a fractional-order system.

	c_0	c_1	c_2	c_3	c_4	c_5	c_6
$\alpha = 2$	1	-2	1	0	0	0	0
$\alpha = 1$	1	-1	0	0	0	0	0
$\alpha = 0.5$	1	-0.5	-0.125	-0.0625	-0.0391	-0.0273	-0.0205

Table 1: Grünwald–Letnikov coefficients. For $\alpha = 0.5$ the sequence never ends.

For $0 < \alpha < 1$ the coefficients have two properties that the rest of this document uses (Lemma 1 of the main document):

- every c_k with $k \geq 1$ is negative, and
- they sum to zero, $\sum_{k\geq 0} c_k = 0$, which is $(1-B)^\alpha$ applied to a constant sequence.

Since $c_0 = 1$, the second property says $\sum_{k\geq 1} |c_k| = 1$.

1.3 The GL-FLIF update

Replace dV/dt in Eq. (1) by $D^\alpha V$, approximate it by $\Delta t^{-\alpha}\nabla^\alpha V_n$, and keep only the last L past values, because hardware has a finite memory buffer. Evaluating the leak at the previous step, as forward Euler does, gives

$$\frac{1}{\Delta t^\alpha}\sum_{k=0}^{L} c_k\, V_{n-k} = -\frac{V_{n-1} - V_{\text{rest}}}{\tau_m} + I_n + b.$$

Since $c_0 = 1$, the $k = 0$ term is V_n, and solving for it gives the update the main document calls GL-FLIF:

$$V_n = \Delta t^\alpha\left(-\frac{V_{n-1} - V_{\text{rest}}}{\tau_m} + I_n + b\right) - \sum_{k=1}^{L} c_k\, V_{n-k}. \tag{5}$$

The threshold rule is applied afterwards, exactly as for Eq. (2). For $\alpha = 1$ only $c_1 = -1$ survives, the last term becomes $+V_{n-1}$, and Eq. (5) is Eq. (2).

Reading the update as a weighted average. Because $c_k < 0$ for $k \geq 1$, the last term of Eq. (5) is $+\sum_{k=1}^{L} |c_k|\, V_{n-k}$: a weighted average of the past potentials, with weights that decay like $k^{-1-\alpha}$. The new potential is that average plus the current drive. An integer-order neuron remembers only its previous value. A fractional neuron remembers a smeared-out version of its whole history.

What truncation does. With all past values the weights sum to 1. With only L of them they sum to $1 - \varepsilon_L$, where

$$\varepsilon_L = \sum_{k=0}^{L} c_k = \frac{\Gamma(L+1-\alpha)}{\Gamma(1-\alpha)\,\Gamma(L+1)} \approx \frac{L^{-\alpha}}{\Gamma(1-\alpha)} \tag{6}$$

is what the discarded tail would have contributed. The missing weight acts like an extra leak that pulls the potential toward 0. For $\alpha = 0.5$ and $L = 6$, the partial sum of Table 1 gives $\varepsilon_6 = 0.226$. For $L = 64$, $\varepsilon_{64} = 0.070$.

1.4 The network

A reservoir is N such neurons, indexed $i = 1, \ldots, N$, connected by a fixed random weight matrix W. The entry W_{ij} is the strength of the connection from neuron j to neuron i. A spike from neuron j reaches its targets one step later, and external inputs $u_{m,n}$ enter through a second matrix W^{in}. The input current of neuron i is therefore

$$I_{i,n} = \sum_j W_{ij}\, s_{j,n-1} + \sum_m W^{\text{in}}_{im}\, u_{m,n}. \tag{7}$$

In reservoir computing, W and W^{in} are never trained. Only a linear *readout* of the neurons' states is fitted to the task. All the simulations in this supplement use one neuron with $\Delta t = 1$, $\alpha = 0.5$, $L = 64$, $\tau_m = 10$ steps, $V_{\text{rest}} = V_{\text{reset}} = 0$, $V_{\text{th}} = 1$, $b = 0$ and no refractory period. Its input is $I_n = \mu + \sigma\xi_n$, with ξ_n independent standard Gaussians and $\sigma = 0.1$.

2 Why the model is not linear

The analysis in the main document needs a *linear* system, because only then can eigenvalues, superposition and transfer functions be used. The network above has three obstacles.

1. **The output is a step function of the state.** At each step $s = H(V - V_{\text{th}})$, where H is the Heaviside step function. Its derivative is zero everywhere except at the threshold, where it does not exist. A Taylor expansion of H carries no information.
2. **The reset makes the potential jump.** Figure 1 shows the actual potential V dropping to V_{reset} at every spike. A fractional neuron is pulled most of the way back on the very next step, because the weighted average over its past still contains the high pre-spike values. Either way, V is not a smooth function of the input.
3. **Neurons talk with binary pulses.** Neuron i receives $s_{j,n-1} \in \{0, 1\}$, not a smoothly varying signal.

The linearization removes these obstacles with three ideas. Change the state variable so the jumps disappear (Section 3). Average the output so the step becomes smooth (Section 5). Then take an ordinary tangent-line approximation around a typical operating point (Sections 4 and 5).

3 Step 1: the free potential

The *free potential* V^f is what the potential would be if the threshold and reset were switched off. It obeys the same update, Eq. (5), with the same input I_n, and nothing else:

$$V^f_n = \Delta t^\alpha \left(-\frac{V^f_{n-1} - V_{\text{rest}}}{\tau_m} + I_n + b \right) - \sum_{k=1}^{L} c_k\, V^f_{n-k}. \tag{8}$$

This is a linear difference equation with constant coefficients, so V^f is a *linear filter* of the input. If the input is a sum $I = I^{(1)} + I^{(2)}$, the free potential is the sum of the free potentials each part would produce. (The constant terms $V_{\text{rest}}/\tau_m + b$ can be treated as part of the input.) The jumps of obstacle 2 are gone.

Two remarks keep this in perspective.

- V^f is computed alongside the real network. It is driven by the real spikes of the other neurons through Eq. (7), and it does not change the network. It is a bookkeeping variable, and you can compute it from a recorded run.

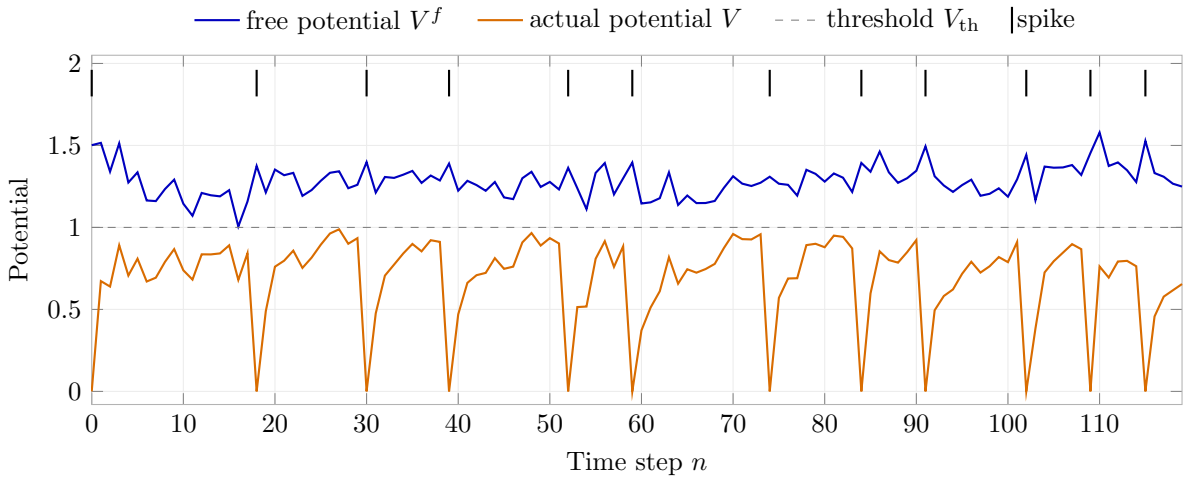


Figure 1: The actual potential V and the free potential V^f of the same neuron, driven by the same input ($\mu = 0.22$). V drops to the reset value at each spike and is pulled back toward its past values on the next step. V^f never resets; it is the input passed through the neuron's filter, and it stays above threshold here because the mean input exceeds the rheobase.

- V and V^f differ only by the effect of past resets. Each reset removes some potential, and that removal is then carried forward by the same linear filter. Section 9 of the main document writes this as an exact identity. In Figure 1, V sits about 0.4 to 0.6 below V^f between spikes: that gap is the accumulated reset deficit.

4 Step 2: the operating point

A tangent line needs a point of tangency. Here that point is the neuron's *operating point*: its average behaviour when the network is running normally. It has three parts.

- The *mean input* $\bar{I}_j$, the time average of $I_{j,n}$.
- The *mean rate* r_j, the time average of $s_{j,n}$, in spikes per step. Since $s_{j,n}$ is 0 or 1, r_j is also the probability of a spike in a given step.
- The *mean free potential* $\bar{V}_j$, the time average of $V^f_{j,n}$.

The mean free potential from the mean input. For a constant input $\bar{I}$, Eq. (8) settles to a constant $\bar{V}$ (assuming the filter is stable). Substituting $V^f_n = \bar{V}$ for every n:

$$\bar{V} = \Delta t^\alpha \left(-\frac{\bar{V} - V_{\text{rest}}}{\tau_m} + \bar{I} + b \right) - \left(\sum_{k=1}^{L} c_k \right) \bar{V}.$$

By Eq. (6), $\sum_{k=1}^{L} c_k = \varepsilon_L - 1$, so the $\bar{V}$ terms combine to $\varepsilon_L \bar{V}$ on the left. Dividing by Δt^α and collecting:

$$\bar{V} = G_{\text{DC}} \left(\frac{V_{\text{rest}}}{\tau_m} + \bar{I} + b \right), \qquad G_{\text{DC}} = \left(\frac{1}{\tau_m} + \frac{\varepsilon_L}{\Delta t^\alpha} \right)^{-1}. \tag{9}$$

G_{DC} is the *DC gain* of the filter: a constant change ΔI in the input moves the free potential by $G_{\text{DC}}\,\Delta I$. With the full history, $\varepsilon_L = 0$ and $G_{\text{DC}} = \tau_m$, the same as the integer-order neuron: a fractional derivative of a constant is zero, so the order α does not change where the neuron settles. With truncation, ε_L adds leak. For the simulated neuron, $G_{\text{DC}} = (0.1 + 0.070)^{-1} = 5.87$ instead of 10. Because Eq. (8) is linear, the time average of V^f obeys Eq. (9) even when the

input fluctuates. In the simulation with $\mu = 0.22$, the measured mean of V^f is 1.2912 and $G_{\mathrm{DC}}\mu = 1.2912$.

5 Step 3: a smooth spike response

5.1 Averaging turns a step into a smooth curve

Obstacle 1 disappears once we stop asking whether a spike happens and ask how *often* it happens. A one-line calculation shows why averaging helps. Suppose that at some step the potential is $v + \xi$, where v is fixed and ξ is random noise with density p and cumulative distribution F. The expected output is

$$\mathbb{E}[s] = \mathbb{E}\big[H(v + \xi - V_{\mathrm{th}})\big] = \Pr(\xi \geq V_{\mathrm{th}} - v) = 1 - F(V_{\mathrm{th}} - v), \qquad \frac{d\,\mathbb{E}[s]}{dv} = p(V_{\mathrm{th}} - v).$$

The step function has become a smooth, increasing curve whose slope is the noise density. For Gaussian noise of standard deviation σ it is the normal CDF, with maximum slope $1/(\sigma\sqrt{2\pi})$ at threshold. As $\sigma \to 0$ the curve steepens back into the step. The same idea applies over time: a neuron in a driven network sees a fluctuating input, and its *average* output is a smooth function of its *average* input.

5.2 The f–I curve

The *f–I curve* $r = f(\bar{I})$ gives a neuron's firing rate (f for frequency) as a function of its mean input. The input level at which a noiseless neuron starts firing is the *rheobase*. For the simulated neuron it is where the settled potential just reaches threshold, $G_{\mathrm{DC}}\,\bar{I} = V_{\mathrm{th}}$, so $\bar{I} = 1/5.87 = 0.170$.

Without noise the curve has an infinite slope at the rheobase. For the integer-order LIF neuron this is a short calculation. Take $V_{\mathrm{rest}} = V_{\mathrm{reset}} = 0$, $b = 0$ and constant input $I > V_{\mathrm{th}}/\tau_m$. Starting from reset, $V(t) = I\tau_m(1 - e^{-t/\tau_m})$ reaches threshold at

$$T(I) = \tau_m \ln \frac{I\tau_m}{I\tau_m - V_{\mathrm{th}}}, \qquad f(I) = \frac{1}{T(I)}, \qquad f'(I) = -\frac{T'(I)}{T(I)^2}.$$

As I decreases to V_{th}/τ_m, $T'(I) \approx -\tau_m^2/(I\tau_m - V_{\mathrm{th}}) \to -\infty$ while T^2 grows only like a squared logarithm, so $f'(I) \to +\infty$. In discrete time, a noiseless neuron's period is a whole number of steps, so its rate can only take values $1/1, 1/2, 1/3, \ldots$ at constant input, and its curve is a staircase. Figure 2 shows both effects, and also shows that input noise smooths them away. The main document uses this *noise-averaged* curve, with the noise level a neuron actually experiences in the running network.

5.3 The tangent line, in units of the free potential

The slope $f'(\bar{I}_j)$ of the noise-averaged curve is the *f–I gain*: extra spikes per step per unit of extra mean input. The linear model is written in terms of the free potential rather than the input, so the slope has to be converted. By Eq. (9), $d\bar{V}_j/d\bar{I}_j = G_{\mathrm{DC}}$, and the chain rule gives the gain per unit of free potential:

$$\gamma_j = \frac{dr_j}{d\bar{V}_j} = \frac{dr_j/d\bar{I}_j}{d\bar{V}_j/d\bar{I}_j} = \frac{f_j'(\bar{I}_j)}{G_{\mathrm{DC}}}. \tag{10}$$

The first-order Taylor expansion of the spike output around the operating point is then

$$s_{j,n} \approx r_j + \gamma_j \left(V_{j,n}^f - \bar{V}_j\right). \tag{11}$$

This is the linearization of the main document. Three things about it are easy to miss.

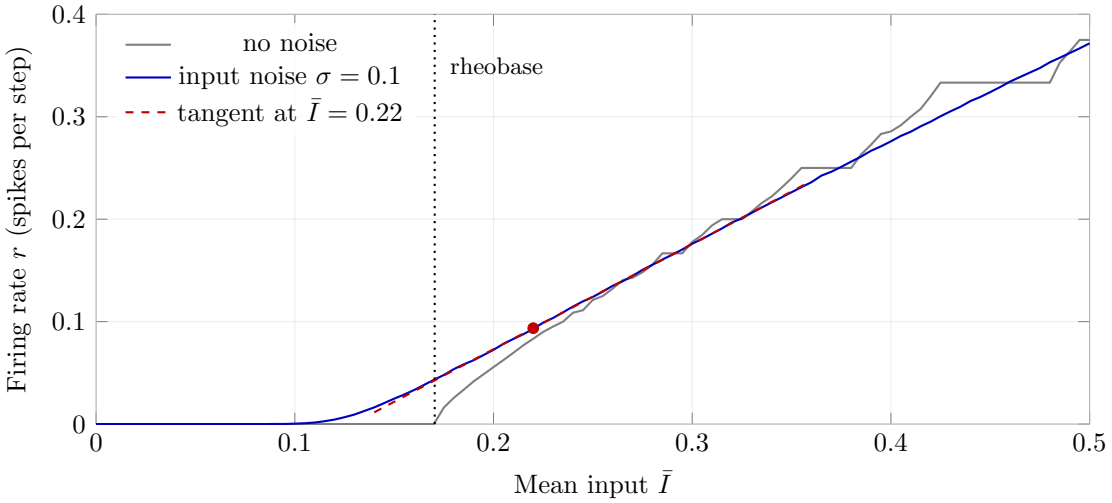


Figure 2: Firing rate against mean input for the simulated neuron. Without noise the curve is a staircase that starts abruptly at the rheobase. With input noise it is smooth, and above the rheobase it is nearly straight: the tangent line at $\bar{I} = 0.22$ stays within a few percent of it from 0.16 up to the edge of the plot. Below the rheobase the rate tails off toward zero and the tangent fails.

- **The "≈" is about averages.** The left side is 0 or 1. The right side is a smooth number near r_j. What Eq. (11) claims is that the right side is the neuron's expected output. The actual output equals it plus a zero-mean "spike noise" that the linear model drops.
- **γ_j is a slow, or DC, gain.** It was computed from mean input against mean rate, so it describes how the rate follows *slow* changes in V^f. Section 6 shows that fast changes get through more strongly.
- **Silent neurons drop out.** A neuron far below its rheobase has $r_j = 0$ and $\gamma_j = 0$. It sends nothing, and the linear model ignores it. A neuron just below its rheobase has a small but nonzero γ_j on the noise-averaged curve, and a fault elsewhere can push it into firing.

6 Measuring r_j, γ_j and $\bar{V}_j$

The mean rate and mean free potential are plain time averages over a recorded fault-free run, after discarding the initial transient. $\bar{V}_j$ needs V_j^f, which is obtained by replaying the recorded input $I_{j,n}$ through Eq. (8).

The gain by sweeping the mean. This is a numerical derivative. Drive a single copy of the neuron with input μ + (noise), with the noise matched to what neuron j sees, and measure the rate at $\mu = \bar{I}_j \pm h$. The central difference

$$f'(\bar{I}_j) \approx \frac{f(\bar{I}_j + h) - f(\bar{I}_j - h)}{2h}$$

then gives γ_j through Eq. (10). The usual finite-difference trade-off applies. If h is too small, the statistical noise in the measured rates swamps the difference. If h is too large, the curvature of f biases it. For the simulated neuron at $\bar{I} = 0.22$, $h = 0.02$ gives $f' = 1.027$ and $\gamma = 1.027/5.87 = 0.175$.

The gain depends on how fast the input changes. To check that 0.175 is right, drive the neuron with $\mu(t) = 0.22 + 0.02\sin(2\pi t/P)$ plus the same noise, and measure how much the rate oscillates per unit of oscillation in V^f. Figure 3 shows the result against frequency $1/P$. For slow oscillations, with periods of a thousand steps or more, the gain is close to 0.176, matching Eq. (10). For faster oscillations it is larger: twice as large at a period of 100 steps, and seven times as large at 10 steps.

The reason is the reset. When V^f rises slowly, the neuron fires more, and every extra spike's reset lowers V for a long time afterwards, through the same memory. That suppresses further spikes and cancels much of the increase. When V^f rises briefly, the extra spikes happen before the cancellation catches up. A fractional neuron spreads the cancellation over its long memory, so its gain changes over a much wider range of frequencies than an integer-order neuron's. In Figure 3 the $\alpha = 1$ neuron is flat down to periods of about 300 steps, and the $\alpha = 0.5$ neuron is not.

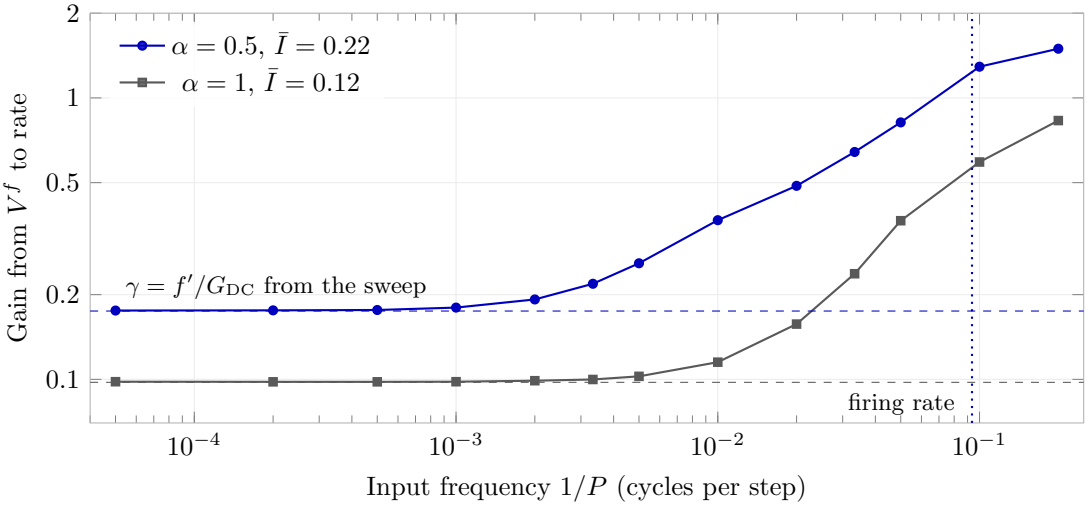


Figure 3: How strongly the firing rate follows an oscillation in the free potential, against the oscillation's frequency. Dashed lines: the slow gain γ of Eq. (10), measured by sweeping the mean input. The measured gain matches it for slow oscillations and grows for fast ones, well below the firing rate (dotted, for $\alpha = 0.5$).

The gain by regression, and why it differs. The main document also suggests a second method: regress the recorded spikes on the free potential and take the slope,

$$\hat{\gamma}_j = \frac{\mathrm{Cov}(s_j, V_j^f)}{\mathrm{Var}(V_j^f)},$$

which is the slope of the best-fitting straight line through the points $(V_{j,n}^f, s_{j,n})$. It is cheap, because it needs only a recorded run. But it weights each frequency by how much of it is present in V^f, and a neuron driven by fluctuating input has plenty of fast fluctuation. In the simulation the regression gives $\hat{\gamma} = 0.84$ for $\alpha = 0.5$, against the slow gain 0.175, and 0.19 against 0.098 for $\alpha = 1$. So the step-by-step regression measures a different, faster gain, not γ_j of Eq. (10).

The gain by windowed regression. The fix is to remove the fast fluctuations before regressing. Split the recorded run into consecutive windows of W steps, average s_j and V_j^f over each window, and regress the window averages on each other. Averaging over a window is a low-pass filter, so only fluctuations slower than about W steps survive, and Figure 3 says those are exactly the

ones whose gain is γ_j. In the simulation, with $\alpha = 0.5$, the slope falls from 0.84 at $W = 1$ to 0.19 at $W = 100$ and 0.1765 at $W = 1000$, against $\gamma = 0.175$ from the sweep. For $\alpha = 1$, $W = 100$ already gives 0.099 against 0.098. A recorded run of 2×10^5 steps, which is 200 windows of 1000 steps, gave estimates within about 5% of γ. This is the golden-run method the main document recommends. It needs no extra experiment, and on hardware it automatically includes truncation and quantization.

7 Step 4: assembling the linear network

7.1 One neuron's equation

Insert the linearized spikes, Eq. (11), into neuron i's input, Eq. (7). Because spikes arrive one step late, the linearization is applied at step $n-1$:

$$\sum_j W_{ij}\, s_{j,n-1} \approx \sum_j W_{ij}\gamma_j\, V^f_{j,n-1} + \sum_j W_{ij}\left(r_j - \gamma_j \bar{V}_j\right).$$

The first sum depends on the other neurons' free potentials. The second is a constant. Substituting into the free-potential update, Eq. (8), and splitting the leak as $-(V^f_{i,n-1} - V_{\text{rest}})/\tau_m = -V^f_{i,n-1}/\tau_m + V_{\text{rest}}/\tau_m$ gives, inside the parentheses,

$$\underbrace{-\frac{V^f_{i,n-1}}{\tau_m} + \sum_j W_{ij}\gamma_j\, V^f_{j,n-1}}_{\text{previous free potentials}} + \underbrace{\sum_m W^{\text{in}}_{im}\, u_{m,n}}_{\text{external input}} + \underbrace{\frac{V_{\text{rest}}}{\tau_m} + b + \sum_j W_{ij}\left(r_j - \gamma_j \bar{V}_j\right)}_{\text{constant}}.$$

Every term is now either a constant or a linear function of free potentials and inputs. Neuron i no longer needs anyone's spikes, only their free potentials.

7.2 A two-neuron example

For two neurons connected to each other but not to themselves, with $W = \left(\begin{smallmatrix} 0 & W_{12} \\ W_{21} & 0 \end{smallmatrix}\right)$, the two equations are

$$V^f_{1,n} = \Delta t^\alpha \left(-\tfrac{1}{\tau_m} V^f_{1,n-1} + W_{12}\gamma_2\, V^f_{2,n-1} + \textstyle\sum_m W^{\text{in}}_{1m} u_{m,n} + \beta_1\right) - \sum_{k=1}^{L} c_k V^f_{1,n-k},$$
$$V^f_{2,n} = \Delta t^\alpha \left(W_{21}\gamma_1\, V^f_{1,n-1} - \tfrac{1}{\tau_m} V^f_{2,n-1} + \textstyle\sum_m W^{\text{in}}_{2m} u_{m,n} + \beta_2\right) - \sum_{k=1}^{L} c_k V^f_{2,n-k},$$

with constants $\beta_1 = V_{\text{rest}}/\tau_m + b + W_{12}(r_2 - \gamma_2\bar{V}_2)$ and $\beta_2 = V_{\text{rest}}/\tau_m + b + W_{21}(r_1 - \gamma_1\bar{V}_1)$. The coefficients of the previous free potentials form the matrix

$$A = \begin{pmatrix} -1/\tau_m & W_{12}\gamma_2 \\ W_{21}\gamma_1 & -1/\tau_m \end{pmatrix} = -\frac{1}{\tau_m}\,\text{Id} + \underbrace{\begin{pmatrix} 0 & W_{12} \\ W_{21} & 0 \end{pmatrix}\begin{pmatrix} \gamma_1 & 0 \\ 0 & \gamma_2 \end{pmatrix}}_{\tilde{W} = W\,\text{diag}(\gamma)}.$$

Each column of W is scaled by the gain of the neuron that sends along it: a connection from a neuron that barely responds carries almost nothing.

7.3 The general form

Stack the free potentials into a vector $x_n = (V^f_{1,n}, \ldots, V^f_{N,n})$. The coefficients c_k are the same numbers for every neuron, so the memory sum stacks directly, and the N equations become one:

$$x_n = \Delta t^\alpha \left(A\, x_{n-1} + B\, u_n + \beta\right) - \sum_{k=1}^{L} c_k\, x_{n-k},$$
$$A = -\tfrac{1}{\tau_m}\mathrm{Id} + \tilde{W}, \quad \tilde{W} = W\,\mathrm{diag}(\gamma), \quad B = W^{\mathrm{in}}, \quad \beta = \left(\tfrac{V_{\mathrm{rest}}}{\tau_m} + b\right)\mathbf{1} + W\left(r - \mathrm{diag}(\gamma)\,\bar{V}\right), \tag{12}$$

where $\mathbf{1}$ is the all-ones vector. This is the *linear fractional-order reservoir* (linear FOR) of the main document. As $\Delta t \to 0$ with the memory $L\Delta t \to \infty$, the left side becomes a GL derivative and Eq. (12) becomes $D^\alpha x = Ax + Bu + \beta$. Everything the main document derives (fault responses, recovery times, stability) comes from the eigenvalues of A and this one equation.

8 When can the approximation be trusted?

The only approximation is Eq. (11). Everything after it is exact algebra. Eq. (11) can fail in four ways.

- **Too large a change.** A tangent line is accurate only where the curve is nearly straight. In Figure 2 the noisy curve is nearly straight above the rheobase, so the tangent at $\bar{I} = 0.22$ stays within a few percent of it from 0.16 upward. Below the rheobase the rate tails off toward zero, and the tangent is 30% off at 0.14. For a fault, the relevant change is the shift in mean input that the fault causes, $-W_{ij}r_j$ for a target of a removed neuron j. It must not carry the neuron across that bend.
- **Too fast a change.** As Figure 3 shows, a single static γ_j is right for slow changes and too small for fast ones. Results about slow quantities, such as the shift in mean operating point after a permanent fault, are safest. Results that depend on fluctuations at all frequencies, such as memory capacity and information measures, inherit an error from the frequency dependence of the gain, and that error is larger at low α.
- **Spike noise.** The linear model keeps the expected output and drops the 0/1 fluctuations around it. In the full network those fluctuations act as extra noise on every neuron.
- **Neurons that switch on or off.** The tangent line of a silent neuron is flat, so the linear model cannot predict a fault pushing it across its rheobase. Section 6 of the main document treats these switches separately.

Each of these can be checked from simulation. Compare the linear model's prediction with the spiking network's response to the same fault, and look at which of the four mechanisms explains the difference.

Symbols

Symbol	Meaning
$V_{i,n}$	actual membrane potential of neuron i at step n (resets at each spike)
$V^f_{i,n}$, x_n	free potential (no threshold or reset); x_n is the vector of all of them
$s_{i,n}$	spike output, 0 or 1
$I_{i,n}$, $u_{m,n}$	input current of neuron i; external input signal m
τ_m, V_{rest}, b	leak time constant, resting potential, bias
V_{th}, V_{reset}	threshold and reset potentials
α, c_k, L	fractional order, GL coefficients $(-1)^k\binom{\alpha}{k}$, memory length
ε_L	weight lost by keeping only L past values; acts as extra leak
G_{DC}	DC gain of the membrane filter, $(1/\tau_m + \varepsilon_L/\Delta t^\alpha)^{-1}$
W, W^{in}	recurrent and input weights; W_{ij} is from neuron j to neuron i
$\bar{I}_j$, r_j, $\bar{V}_j$	operating point: mean input, mean rate, mean free potential
f_j, f'_j	noise-averaged f–I curve and its slope
γ_j	slow gain from free potential to rate, $f'_j(\bar{I}_j)/G_{\mathrm{DC}}$
$\tilde{W}$, A, B, β	matrices and constant of the linear FOR, Eq. (12)